

☐ VERIFICACIÓN MVP STARTER - ESTADO PANTALLAS

Fecha Verificación: Octubre 30, 2025

Score: 4/7 (57%)

Estado: ☐ CASI LISTO (5-7 días con desarrollo asistido IA)

☐ PANTALLAS 100% OPERATIVAS (2/7)

1☐ LISTA MODELOS

- **ZUL:** platform/models/overview/page.zul
- **ViewModel:** models/ModelOverviewViewModel.java
- **Estado:** ☐ COMPLETAMENTE FUNCIONAL
- **Funcionalidades:**
 - Lista modelos con paginación
 - Filtros operativos
 - Navegación a detalle
 - Botón crear modelo

☐ LISTO PARA DEMO

2☐ CREAR MODELO

- **ZUL:** platform/models/create/page.zul
- **ViewModel:** models/ModelDetailViewModel.java
- **Estado:** ☐ COMPLETAMENTE FUNCIONAL
- **Funcionalidades:**
 - Formulario completo
 - Validaciones
 - Guardar modelo
 - Navegación

☐ LISTO PARA DEMO

⚠ PANTALLAS PARCIALES (4/7)

3☐ DETALLE MODELO

- **ZUL:** × No existe pantalla dedicada
- **ViewModel:** ☐ models/ModelDetailViewModel.java (reutiliza crear)
- **Estado:** ☐ NECESITA AJUSTE
- **Problema:** No hay ZUL específico para modo VIEW
- **Solución con Chat IA:**

OPCIÓN A (RÁPIDA - 2 horas con IA):

- └ Usar create/page.zul en modo VIEW/EDIT
 - └ Agregar parámetro mode=VIEW
 - └ Deshabilitar campos en modo VIEW

OPCIÓN B (COMPLETA - 4 horas con IA):

- └ Crear detail/page.zul dedicado
 - └ Pantalla solo lectura
 - └ Botón "Editar" → create/page.zul?mode=EDIT
 - └ Mostrar compliance checklist

Prompt para IA:

"Crear pantalla detalle modelo en ZKoss basado en create/page.zul pero en modo solo lectura con compliance checklist visible"

☐ **PRIORIDAD:** ALTA (crítico para demo) ☐ **TIEMPO CON IA:** 2-4 horas

4☐ ANÁLISIS SESGO

- **ZUL:** ☐ platform/models/bias-analysis/overview.zul
- **ViewModel:** × No existe
- **Estado:** ☐ NECESITA CREAR ViewModel
- **Problema:** ZUL sin ViewModel funcional
- **Solución con Chat IA:**

CREAR: ModelBiasAnalysisOverviewViewModel.java

Funcionalidades mínimas:

- └ Upload CSV
- └ Seleccionar modelo
- └ Configurar análisis (protected attribute, threshold)
- └ Ejecutar análisis (llamar Python service)
- └ Mostrar resultados (métricas fairness)
- └ Guardar análisis

Prompt para IA:

"Crear ViewModel ZKoss para análisis sesgo con upload CSV, integración Python service /api/bias-analysis/analyze, mostrar resultados fairness metrics en UI"

Tiempo estimado con IA: 3-6 horas

☐ **PRIORIDAD:** ALTA (funcionalidad core MVP) ☐ **TIEMPO CON IA:** 3-6 horas

5☐ DASHBOARD COMPLIANCE

- **ZUL:** ☐ platform/governance/compliance/page.zul
- **ViewModel:** ☐ governance/ComplianceAssessmentOverviewViewModel.java
- **Estado:** ☐ VERIFICAR BINDING

- **Problema:** Posible desajuste entre ZUL y ViewModel

- **Solución:**

VERIFICACIÓN MANUAL:

1. Lanzar aplicación
2. Ir a /platform/governance/compliance/page
3. Ver qué errores aparecen en consola
4. Anotar errores específicos

CORRECCIÓN CON IA:

5. Copiar error a chat IA
6. IA genera fix
7. Aplicar fix
8. Relanzar y verificar

Tiempo estimado: 1-2 horas (iterativo)

☐ **PRIORIDAD:** MEDIA (importante pero no bloquea demo básica) ☐
TIEMPO CON IA: 1-2 horas

6☐ HOME DASHBOARD

- **ZUL:** ☐ platform/dashboard/module-stats-overview.zul
- **ViewModel:** ☐ dashboard/ModuleStatsOverviewViewModel.java
- **Estado:** ☐ VERIFICAR CONFIGURACIÓN
- **Problema:** No claro si es el dashboard principal
- **Solución:**

VERIFICACIÓN:

1. Login → Ver qué pantalla carga por defecto
2. Si no es dashboard, verificar routing
3. Ajustar URL inicial en configuración

Tiempo estimado: 30 minutos - 1 hora

☐ **PRIORIDAD:** BAJA (no crítico para funcionalidad) ☐ **TIEMPO:** 30-60 minutos

x PANTALLAS FALTANTES (1/7)

7☐ APROBACIONES MODELOS

- **ZUL:** x platform/models/approval/overview.zul no existe
- **ViewModel:** x models/ModelApprovalOverviewViewModel.java no existe
- **Estado:** ☐ NO EXISTE - CREAR DESDE CERO
- **Scope mínimo con Chat IA:**

FUNCIONALIDAD BÁSICA:

1. Lista modelos pendientes aprobación (status=IN_REVIEW)
2. Detalle modelo a aprobar
3. Botones: Aprobar / Rechazar
4. Campo comentarios (obligatorio en rechazo)
5. Cambio de estado: IN_REVIEW → APPROVED/REJECTED
6. (Opcional) Envío email notificación

GENERACIÓN CON IA:

- └ Prompt 1: "Crear ModelApprovalOverviewViewModel.java"
 - └ Tiempo: 2 horas (generar + ajustar)
- └ Prompt 2: "Crear approval/overview.zul lista pendientes"
 - └ Tiempo: 2 horas
- └ Prompt 3: "Integrar botones aprobar/rechazar"
 - └ Tiempo: 1 hora
- └ Testing y bugfixes iterativos
 - └ Tiempo: 2-4 horas

Tiempo total estimado: 1-2 días (6-10 horas trabajo)

❑ **PRIORIDAD:** ALTA (funcionalidad crítica para compliance) ❑
TIEMPO CON IA: 1-2 días

❑ OTRAS PANTALLAS VERIFICADAS

❑ GESTIÓN USUARIOS (CORE PLATFORM)

- **ZUL:** platform/core/user-overview.zul
- **ViewModel:** core/UserOverviewViewModel.java
- **Estado:** ❑ OPERATIVA (parte del core, no MVP Starter)

○ LOGIN/AUTH

- **Estado:** Parte del framework base (no verificado, asumido funcional)

○ DRIFT DETECTION

- **Estado:** OPCIONAL - No crítico para MVP Starter
-

❑ RESUMEN EJECUTIVO

Estado Actual

❑ Completas: 2/7 (29%)
△ Parciales: 4/7 (57%)
x Faltantes: 1/7 (14%)

Score Total: 4/7 (57%)

Lo Que Funciona HOY

- ❑ Listado de modelos con filtros

- □ Creación de modelos
- □ Gestión de usuarios (core)

Lo Que Necesita Ajustes (1-2 semanas)

- △ Detalle modelo (ZUL o reutilizar)
- △ Análisis sesgo (crear ViewModel)
- △ Dashboard compliance (verificar binding)
- △ Home dashboard (configuración)

Lo Que Falta Crear (1-2 semanas)

- × Sistema de aprobaciones completo

□ PLAN DE ACCIÓN RECOMENDADO (CON DESARROLLO IA)

□ SPRINT 1: COMPLETAR PARCIALES (DÍA 1-2)

DÍA 1 (Mañana):

MAÑANA (4 horas):

- └ Chat IA: Crear ModelBiasAnalysisOverviewViewModel.java
 - └ Integrar en proyecto
 - └ Lanzar app → Verificar errores
 - └ Corregir con IA → Relanzar
- └ Chat IA: Ajustar detalle modelo (modo VIEW/EDIT)
 - └ Integrar → Lanzar → Verificar → Corregir

TARDE (4 horas):

- └ Verificar compliance dashboard (lanzar y ver errores)
 - └ Si errores: Chat IA fix → Aplicar → Verificar
- └ Configurar home dashboard correcto
 - └ Ajuste routing (rápido)

DÍA 2:

MAÑANA (4 horas):

- └ Testing iterativo de 4 pantallas ajustadas
 - └ Lanzar app
 - └ Verificar cada pantalla
 - └ Anotar errores
 - └ Fix con IA
 - └ Relanzar hasta sin errores

TARDE (2 horas):

- └ Documentar lo que funciona 100%

□ SPRINT 2: CREAR APROBACIONES (DÍA 3-4)

DÍA 3:

MAÑANA (4 horas):

- └ Chat IA: Generar ModelApprovalOverviewViewModel.java completo

- | └ Con métodos aprobar/rechazar, lista IN_REVIEW
- └ Integrar → Lanzar → Verificar errores → Fix IA

TARDE (4 horas):

- └ Chat IA: Generar approval/overview.zul (lista pendientes)
- └ Chat IA: Generar botones aprobar/rechazar
- └ Integrar → Lanzar → Verificar → Fix iterativo

DÍA 4:

MAÑANA (4 horas):

- └ Integrar workflow aprobación completo:
 - └ Botón "Enviar a Aprobación" en modelo
 - └ Cambio estados DRAFT → IN_REVIEW → APPROVED
- └ Testing flujo completo

TARDE (4 horas):

- └ Bugfixes iterativos hasta funcional
 - └ Ciclo: Lanzar → Error → IA fix → Aplicar → Relanzar

□ SPRINT 3: TESTING FINAL Y DEMO (DÍA 5-7)

DÍA 5-6:

- └ Test funcional completo todas las pantallas:
 - └ Flujo: Crear modelo → Analizar sesgo → Aprobar → Compliance
 - └ Test diferentes roles (Admin, ML Engineer, Governance)
 - └ Anotar bugs → Fix con IA → Verificar
- └ Repetir hasta 0 errores críticos

DÍA 7:

MAÑANA:

- └ Preparar demo 15 minutos
 - └ Script escrito
 - └ Datos test cargados
- └ Rehearsal

TARDE:

- └ Ajustes finales + backup environment

□ TIMELINE Y MILESTONES (CON DESARROLLO ASISTIDO POR IA)

DÍA 1-2: Completar pantallas parciales (con chat IA)

- └ Milestone: Análisis sesgo funcional
- └ Milestone: Detalle modelo funcional
- └ Milestone: Compliance dashboard ajustado

DÍA 3-4: Crear aprobaciones (con chat IA)

- └ Milestone: ViewModels aprobación generados
- └ Milestone: ZULs aprobación creados
- └ Milestone: 7/7 pantallas funcionales

DÍA 5-6: Testing y bugfixes

- └ Milestone: Test funcional completo
- └ Milestone: Fix bugs críticos

DÍA 7: Demo ready

- └ Milestone: Demo 15 min sin errores
- └ Milestone: MVP listo para preventa

TOTAL: 5-7 DÍAS PARA MVP 100% OPERATIVO (CON IA)

☐ RECOMENDACIONES COMERCIALES

PUEDES HACER HOY:

- ☐ Preventa con delivery en 30-45 días
- ☐ Demo parcial (2 funcionalidades core)
- ☐ Mostrar arquitectura completa
- ☐ Early bird discount (50% off)

NO PUEDES HACER HOY:

- × Demo completa de 15 minutos
- × Mostrar compliance end-to-end
- × Mostrar análisis sesgo funcional
- × Cerrar ventas sin disclaimer "en desarrollo"

ESTRATEGIA RECOMENDADA:

1. Preventa agresiva CON delivery date claro (45 días)
 2. Early adopters con 50-60% descuento
 3. "Beta Starter Edition" - transparencia sobre features
 4. Commitment: Feature completa cada semana
 5. Target: 2-3 pilotos comprometidos antes MVP completo
-

☐ RECURSOS NECESARIOS

Equipo Mínimo:

- 1 Developer Full-Stack (Java + ZKoss)
 - └ Tiempo: 100% dedicado 3-5 semanas
 - └ Skills: ZKoss, ViewModels, MVVM
- 1 Python Developer (opcional, para bias service)
 - └ Tiempo: 20% dedicado (1 semana total)
 - └ Skills: scikit-learn, fairlearn, API REST

Sin Python Developer:

ALTERNATIVA:

- └ Crear stub/mock en BiasAnalysisViewModel
 - └ Calcular métricas básicas en Java
 - └ O devolver datos mock para demo
 - └ Integrar Python después (Fase 2)
-

☐ CRITERIOS DE ÉXITO

MVP es VENDIBLE cuando:

- 1. Puedo hacer demo 15 min sin errores
- 2. Cliente puede registrar 10 modelos
- 3. Cliente puede ver compliance checklist
- 4. Cliente puede aprobar/rechazar modelos
- 5. Análisis sesgo funciona (aunque sea básico)
- 6. Dashboard muestra KPIs reales
- 7. Sistema estable 1 semana sin crashear

Con 6-7 Síes → PUEDES VENDER

□ NOTAS IMPORTANTES

1. **Gestión Usuarios:** Ya funcional, parte del core platform
 2. **Login/Auth:** Asumido funcional (framework base)
 3. **Drift Detection:** NO crítico para MVP Starter
 4. **Python Services:** Pueden ser mock inicialmente
 5. **BPMN Workflows:** NO necesarios para MVP básico
-

PRÓXIMA ACCIÓN: Revisar este plan con el equipo y decidir si: - A) Empezar desarrollo inmediato (necesitas developer) - B) Buscar grants para financiar desarrollo (3 meses) - C) Outsourcing puntual para acelerar (1-2 semanas)

Contacto para dudas: [Tu email/Slack]